Final Project Completion Report

Chiron Pro: Language Extensions and VS Code Extension

Dnyanesh Hemendra Shinde (230390)
Nikhilesh Joshi (230700)
Yatharth Dangi (231197)

GitHub Repository: `https://github.com/EMPEROR82/ChironIITK-CS335`
Commit Identifier: `d55913ad778909310cc0088937858d3f5f751cd8`

Note: The above repository is the development repository forked from the original Chiron Project repository, but the final VS Code extension was released using the below repostiory:

VS Code Extension Repository: `https://github.com/EMPEROR82/ChironPro`
Commit Identifier: `12f962784839ddc2d4cd59142594f8420cb20468`
Extension Release: d97231e14f8c40dc09603919fbd2490f2f3ebba9
Extension Link: `https://marketplace.visualstudio.com/items?itemName=EMPEROR82.chiron-extension`

1. Introduction

This project presents **Chiron Pro**, an enhanced programming ecosystem for the Chiron language that combines language-level improvements with modern IDE support.

The work was carried out in two major stages:

- Extending the core language with new constructs such as functions and arrays
- Developing a Visual Studio Code extension using the Language Server Protocol (LSP)

The final system provides an integrated environment offering syntax highlighting, semantic diagnostics, intelligent autocompletion, hover-based insights, and interactive execution support.

2. Motivation

The original Chiron language lacked both expressive programming constructs and developer tooling. This resulted in:

- Difficulty in writing modular and reusable programs
- Limited debugging capabilities due to absence of real-time feedback
- Poor code comprehension due to lack of semantic insights

To address these challenges, this project enhances the language and builds an IDE layer that enables:

- Structured programming through functions and arrays
- Continuous feedback during development
- Improved understanding via visualization and metadata

3. Methodology

The system was developed in a structured two-phase pipeline:

Phase 1: Language and Compiler Enhancements

- Grammar rules were extended to support new constructs
- AST representation was expanded
- Semantic analysis was updated for scope handling and validation

Phase 2: IDE Development using LSP

- Implemented a VS Code client in TypeScript
- Built a language server handling LSP requests
- Integrated compiler logic into IDE features

4. System Architecture

The architecture follows a client-server model:

- **Client:** Handles UI and user interactions in VS Code
- **Server:** Processes requests such as completion, hover, and diagnostics

Communication is achieved using JSON-RPC over standard input/output streams, ensuring loose coupling and extensibility.

5. Phase 1: Language Extensions

5.1 Functions

Support for user-defined functions was introduced, including:

- Multiple parameters
- Return statements
- Recursive and nested calls

Semantic analysis ensures proper scoping and argument handling.

5.2 Arrays

The language now supports:

- Static array declaration
- Indexed element access and assignment

5.3 Expression Enhancements

Additional operators were incorporated:

- Modulo operator
- Exponentiation operator

5.4 Print and Strings

A print mechanism was introduced to allow runtime output, with support for string arguments to improve debugging and usability.

6. Phase 2: VS Code Extension

6.1 Syntax Highlighting

Syntax highlighting is implemented using a declarative TextMate grammar. Token categories such as keywords, identifiers, literals, and operators are defined through regular expressions, allowing VS Code's tokenization engine to apply appropriate styling automatically.

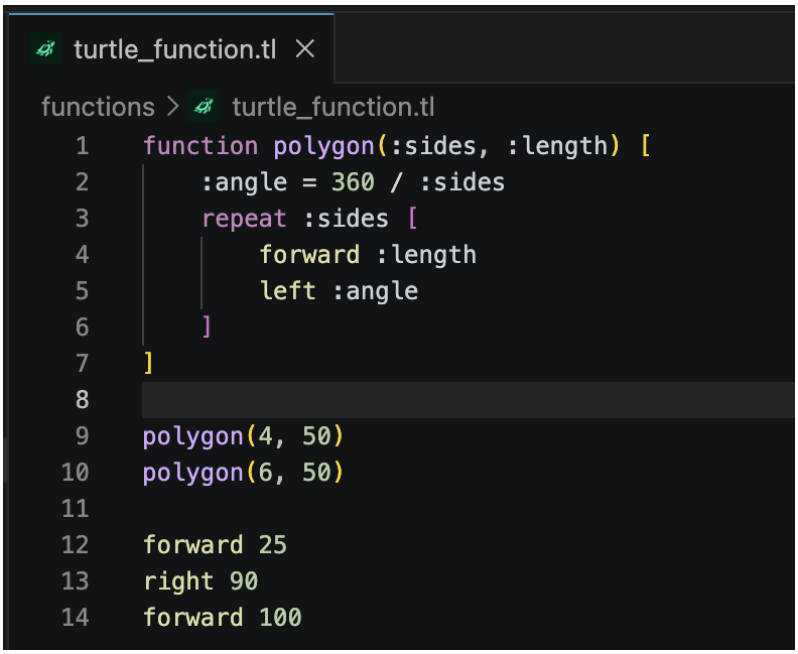
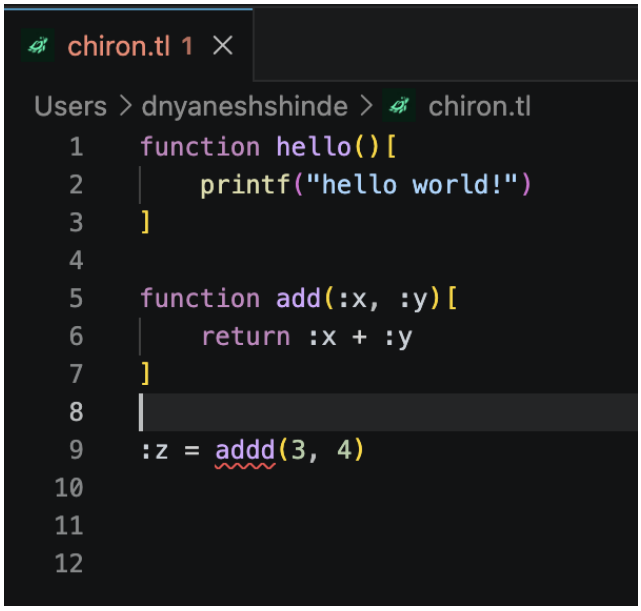


Figure 1: Syntax highlighting of Chiron code

6.2 Semantic Error Highlighting

The extension performs continuous semantic analysis by invoking the compiler pipeline on every document change. Errors such as undefined variables or incorrect function usage are detected and reported through LSP diagnostics, which are visually rendered as squiggly underlines in the editor.



(a) Incorrect name of function

(b) Use without declaration

Figure 2: Hover information for functions and variables

6.3 Autocompletion

Autocompletion is handled by the language server through the `textDocument/completion` request. The server scans the document to extract symbols such as variables and function names and returns context-aware suggestions. These suggestions are filtered based on scope and prefix matching.

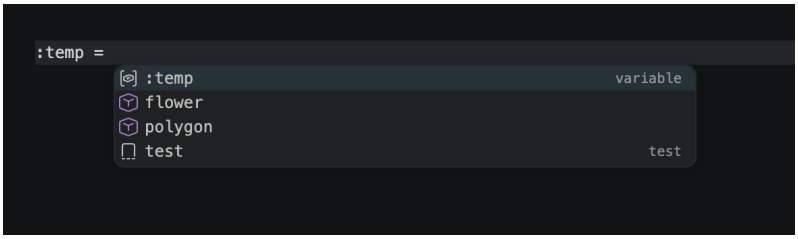


Figure 3: Multiple autocomplete suggestions

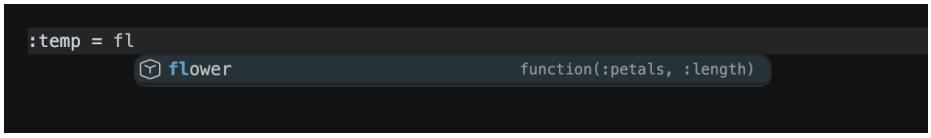


Figure 4: Reduced autocomplete suggestions based on prefix matching

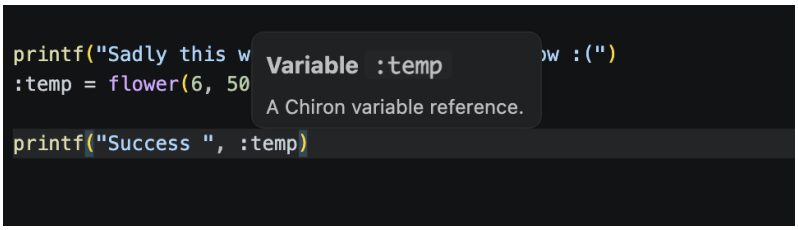
6.4 Hover Information

Hover functionality is implemented using the `textDocument/hover` request. When a user hovers over a symbol:

- Function signatures are displayed
- Variable-related metadata is shown

The information is formatted in Markdown and rendered in the editor.

(a) Hover information for function



(b) Hover information for variable

Figure 5: Hover information for functions and variables

6.5 Go-to-Definition

Navigation support is provided through the `textDocument/definition` request. The server identifies the declaration of a symbol and returns its location, enabling seamless navigation within the codebase.

Figure 6: Function definition jump

6.6 Editor Assistance Features

Additional editor-level enhancements include:

- Automatic bracket pairing
- Smart indentation based on syntax rules
- Recognition of Chiron files with custom icons

Figure 7: Automatic bracket closing

Figure 8: Auto indentation for code blocks

6.7 Run Button Integration

A run command was integrated into the VS Code interface. When triggered:

- The current file is saved automatically
- Users can choose execution modes (with or without variables or launch a web-view to the side)
- Execution is performed through a reusable terminal session

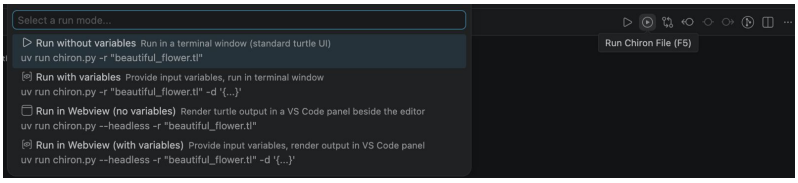


Figure 9: Run options in VS Code

6.8 Webview

An interactive webview component was incorporated to enhance user understanding of program behavior. This enables visualization of execution or structural aspects of the code, improving learning and debugging.

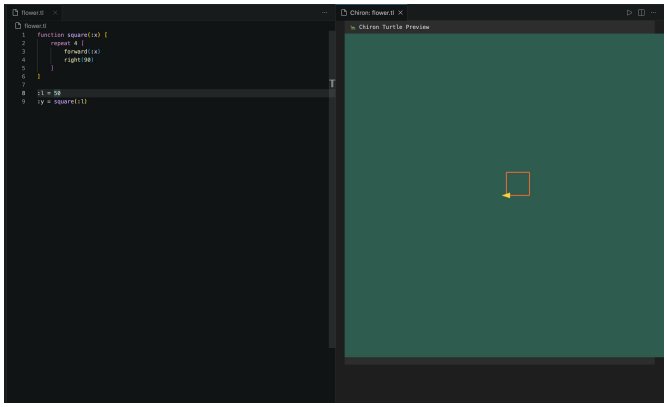


Figure 10: Webview window launched to the side

6.9 Theme and Extension Settings

Support for custom themes and modifying extension setting was also added, allowing users to tailor the visual appearance of the language, thereby improving readability and user experience.

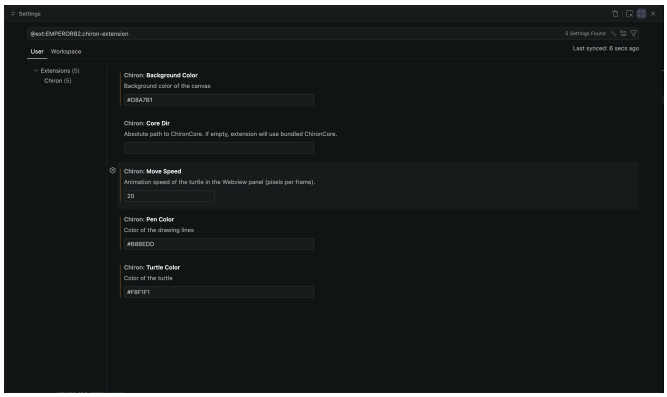


Figure 11: Theme Customization Window

7. Strengths

- **End-to-end integration of language and IDE:** The project successfully delivers a complete pipeline starting from language design and compiler extensions to a fully functional IDE environment. This tight integration ensures that users can both write and execute programs within a unified ecosystem, significantly improving usability.
- **Modular LSP-based architecture:** The adoption of the Language Server Protocol introduces a clean separation between the VS Code client and the language server. This modularity not only simplifies development and debugging but also makes the system scalable and adaptable for future extensions such as debugging support or additional analysis tools.
- **AST-driven extensibility:** The use of an Abstract Syntax Tree as the central representation enables systematic handling of language constructs. This design allows new features (such as hover information, diagnostics, and navigation) to be built on top of a common structure, reducing redundancy and improving maintainability.
- **Real-time feedback loop:** Continuous re-analysis of the program ensures that users receive immediate feedback in the form of diagnostics, autocompletion suggestions, and hover information. This significantly enhances the development experience by enabling faster debugging and iteration.
- **Reuse of compiler infrastructure within the IDE:** Core compiler components such as the parser and semantic analysis modules are reused directly within the language server. This guarantees consistency between compilation and IDE feedback while avoiding duplication of logic. Additionally, integration with the interpreter allows seamless execution support directly from the editor.

8. Limitations

- **Full re-analysis on each edit:** The current implementation performs a complete re-parse and semantic analysis after every document change. While this simplifies implementation, it introduces noticeable latency for larger programs and highlights the need for incremental parsing techniques.
- **Limited error recovery mechanisms:** The compiler's ability to handle malformed or partially incorrect programs is restricted. As a result, diagnostics may become less informative in the presence of cascading errors, affecting the accuracy of editor feedback.
- **Basic autocomplete ranking:** The autocompletion system primarily relies on prefix-based matching and does not incorporate advanced ranking strategies such as frequency-based ordering, context prioritization, or fuzzy matching. Consequently, suggestions may not always align with user intent, especially in cases of minor typographical errors.
- **Lack of advanced IDE capabilities:** While essential features such as navigation and diagnostics are implemented, the extension does not yet support more sophisticated IDE functionalities such as refactoring tools, code actions, or intelligent code generation. These features would further enhance developer productivity and usability.

9. Future Work

- Debugger integration using DAP
- Incremental parsing for performance improvement
- Enhanced autocomplete with ranking and more snippets
- Advanced refactoring support

10. Extension Release on VS Code Marketplace

As part of the final phase of the project, the ChironPro extension was successfully packaged and published on the **Visual Studio Code Marketplace**. This marks the transition of the project from a development prototype to a deployable tool accessible to users.
The extension is available publicly and can be installed directly within VS Code using the extension identifier: `emperor82.chiron-extension`.
The extension was bundled using the VS Code extension packaging tools and published via the official marketplace publishing pipeline.
The release ensures that all implemented features—including syntax highlighting, diagnostics, autocompletion, hover information, and execution support—are available to end users in a seamless manner.

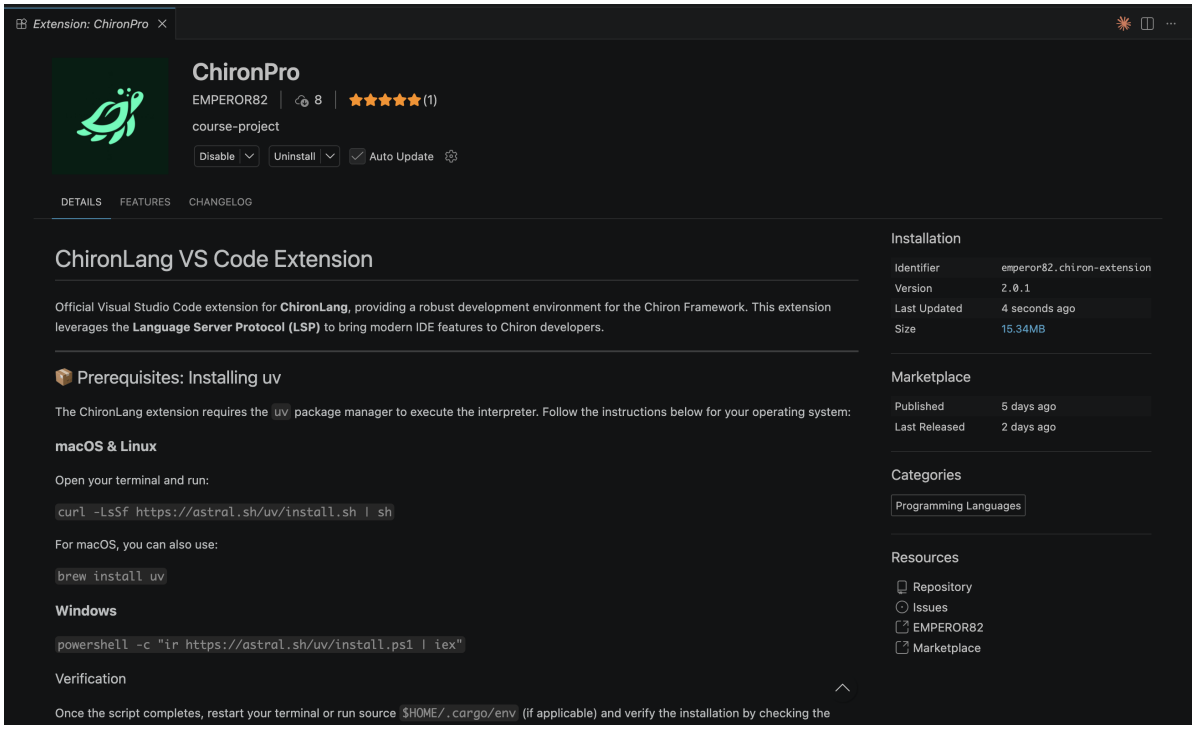


Figure 12: ChironPro extension on VS Code Marketplace

11. Test Cases and Validation

Tests Folder: /ChironCore/Test(ProjectMilestone1)

To verify the correctness of the newly implemented language features, we developed a suite of test programs covering different aspects of the extended Chiron language. These test cases were designed to ensure that the compiler correctly handles functions, parameter passing, recursion, array operations, variable scoping, and the newly introduced operators.

11.1 Function Tests

Several test programs were created to validate the behavior of functions with parameters and return values.

- **simple_function.tl**
  Tests basic function declaration, parameter passing, and returning values.
- **nested_function_calls.tl**
  Verifies that the compiler correctly handles nested function calls and proper evaluation order.
- **recursive_function.tl**
  Tests recursive function calls and ensures that the call stack is managed correctly.

- **scoping_with_functions.tl**
  Validates correct variable scoping between functions and local variables.

- **turtle_function.tl**
  Tests integration of user-defined functions with existing turtle commands in the language.

11.2 Array Tests

The following test program was used to validate the implementation of arrays and indexed access.

- **arrayAccess.tl**
  Tests array declaration, element assignment, and indexed retrieval. This verifies that array memory layout and indexing logic are handled correctly.

11.3 Operator and Expression Tests

To validate the newly added arithmetic operators and expression handling, the following tests were used:

- **misc1.tl** and **misc2.tl**
  Contain mixed arithmetic expressions that test the correct parsing and evaluation of operators including the modulo operator (`%`) and exponentiation operator (`**`).

11.4 Basic Language Feature Tests

Some additional programs were used to confirm that the extended language remains compatible with existing constructs.

- **num.tl**
  Tests numeric variable handling and arithmetic expressions.

- **string.tl**
  Verifies correct handling of string values.

- **var.tl**
  Tests variable declarations and assignments.

- **beautiful_flower.tl**
  A larger program that uses turtle graphics commands to draw a flower. This serves as an end-to-end test to ensure that the compiler continues to correctly execute larger programs after the language extensions.

12. Declaration of AI Usage

AI tools such as ChatGPT and Claude were used to assist in TypeScript implementation (about 60 % of the code). All generated content was reviewed and adapted.

13. References

- VS Code API: `https://code.visualstudio.com/api`

- Language Server Protocol: `https://microsoft.github.io/language-server-protocol/`